SQL itself. The best way to achieve this is to only use parameterized SQL, where placeholders are used to represent dynamic elements of the SQL statement, and the database engine replaces them with user input in a safe way.

The good news is that most ORMs like EF Core always generate parameterized SQL. Listing 13.12 shows the SQL generated when you follow an Edit link in the property manager for a specific property. It contains a parameter named `@__id_0`, which is replaced by the value passed to the `id` parameter in the URL: `property-manager/details?id=1`.

Listing 13.12 The parameterized SQL generated by EF Core

```
SELECT "p"."Id", "p"."Address", "p"."AvailableFrom",
  ➥ "p"."CityId", "p"."CreatorId",
 "p"."DayRate", "p"."Deleted", "p"."MaxNumberOfGuests", "p"."Name",
 "p"."SmokingPermitted", "c"."Id", "c"."CountryId", "c"."Name", "c"."Image"
     FROM "Properties" AS "p"
     INNER JOIN "Cities" AS "c" ON "p"."CityId" = "c"."Id"
     WHERE "p"."Id" = @__id_0     ❶
     LIMIT 1
```

❶ The parameter is named @__id_0.

When strings are passed to parameters, they are treated literally and cannot become part of the resulting SQL statement, which mitigates SQL injection as a vulnerability. If you are writing your own SQL, you must always use parameters to represent user input:

```
select * from cities where name = @name
```

How you go about executing this example will depend on the tool you are using. Dapper accepts the parameters as anonymous objects. The code in the next listing shows an example of the Dapper `QuerySingleAsync` method that takes the SQL statement followed by the parameter values.

Listing 13.13 Passing parameters to an SQL statement using Dapper

```
var sql = "select * from cities where name = @name";
var city = conn.QuerySingleAsync(sql, new {name = Name});
```

EF Core supports executing raw SQL queries via the `FromSqlRaw` method, which takes a `params` object that represents the parameter values. The EF Core equivalent to the parameterized Dapper command is shown in the following listing.